TECHNICAL DOCUMENTATION

CompareX: Analytical Recommendation Engine

A deterministic hybrid recommendation platform combining collaborative filtering with semantic search and LLM-powered explanations. This document serves as the authoritative technical reference for developers and engineering teams integrating with or extending the CompareX platform.

Hybrid Architecture

Deterministic ML models paired with LLM-powered semantic understanding — no hallucination, grounded in real metadata.

Multi-Domain Coverage

Three independent recommendation domains: Anime (SVD), Steam Games (ALS), and Books — each with domain-specific model tuning.

Production-Ready

Deployed on Vercel + Render with MongoDB Atlas and Pinecone vector search. Full auth layer with JWT, CSRF protection, and OAuth.

01

Overview & Architecture

System components, data flow, and technology stack

02

Microservices & Auth

Service boundaries, authentication, and security model

03

Recommendation Flows

Standard ML pipeline and AI-assisted semantic search

04

Setup & Deployment

Local development, environment configuration, and production deployment

ARCHITECTURE

Architecture Overview

CompareX follows a three-tier microservices architecture with strict service boundaries. All client traffic flows through a centralized API Gateway that handles authentication and proxying — the ML service is never directly exposed to external clients. This separation enables independent scaling of compute-heavy inference workloads from lightweight API routing.

Frontend

React SPA (Vite + TailwindCSS) — Browse, Compare, Chat, Login

API Gateway

Express.js — JWT auth, CSRF protection, rate limiting, proxy routing

ML Service

Python FastAPI — domain recommenders, hybrid LLM routing, vector search

Data Layer

MongoDB + Pinecone + pickled ML model artifacts

Technology Stack

Layer	Technology	Role
Frontend	React, Vite, TailwindCSS	UI rendering, state mgmt
Gateway	Express.js, Mongoose	Auth, proxying, rate limiting
ML Service	Python, FastAPI, uvicorn	Inference, embeddings, LLM
Metadata Store	MongoDB Atlas	User data, item metadata
Vector Store	Pinecone (384-dim, Cosine)	Semantic similarity search
Model Artifacts	Pickle files (ALS, SVD)	Offline-trained, fast inference

Design Principles

Gateway as Security Perimeter

The ML service trusts requests forwarded by the gateway unconditionally. No external client ever reaches the inference layer directly.

Deterministic Core, LLM Periphery

Recommendations are always generated by classical ML models. LLMs are used exclusively for intent parsing and explanation — never for item selection.

Graceful Degradation

Constraint relaxation and LLM fallback chains ensure the system continues serving results under model or API failures.

MICROSERVICES

Microservices Structure

Each service maintains a clearly delimited responsibility boundary. The frontend is a pure UI consumer, the gateway owns all authentication and routing logic, and the ML service encapsulates all inference and AI orchestration. This structure enables independent deployment, testing, and scaling of each tier.

Frontend — React SPA

- **Browse Surface:** Domain product discovery with filter controls and paginated results
- **Compare Surface:** Side-by-side item comparison using DomainProductCards
- **Chat Surface:** Streaming conversational interface with inline product cards
- **Login Surface:** Credential form with Google OAuth integration
- **Custom Hooks:** Encapsulate API state, loading states, and error handling across surfaces

API Gateway — Express.js

- **Auth Controllers:** Registration, login, and token refresh endpoints
- **Proxy Controllers:** Forward validated requests to ML service routes
- **JWT Middleware:** Token verification on all protected routes
- **Rate Limiting:** Per-IP request throttling to prevent abuse
- **Mongoose Schemas:** User document modeling with bcrypt password fields

ML Service — Python FastAPI

- **Recommender Routes:** `/recommend/anime`, `/recommend/steam`, `/recommend/books`
- **Assistant Routes:** Chat endpoint with streaming support for conversational queries
- **Domain Services:** `AnimeService`, `SteamService`, `BookService` — each with domain-aware filtering logic
- **HybridLLMClient:** Cascading LLM provider fallback (Gemini → Groq → Claude)

Model Artifacts & Fallbacks

- **ALS Model:** Pickled implicit feedback model for Steam game recommendations based on playtime signals
- **SVD Model:** Pickled matrix factorization model for Anime with explicit user ratings
- **Baseline JSON:** Static fallback item lists loaded when ML model constraints produce no results
- **Constraint Relaxation:** `relax_constraints_and_retry` progressively loosens filters before triggering JSON fallback

SECURITY

Authentication & Security Flow

CompareX implements a layered security model. All credential operations are handled exclusively by the gateway — the ML service operates in a trusted internal network context and never processes credentials directly. Session state is maintained via HttpOnly cookies to eliminate client-side token exposure.

Credential Authentication

User passwords are stored exclusively as `bcrypt` hashes in MongoDB. The gateway compares submitted credentials against stored hashes — plaintext passwords are never persisted or logged at any layer.

OAuth Integration

Google OAuth is integrated via the `@react-oauth/google` package. OAuth token exchange occurs at the gateway, which issues a CompareX session JWT identical to the credential-based flow — OAuth vs. password auth is transparent to the ML service.

Session & Request Security

JWTs are stored in `HttpOnly`, `Secure` cookies, preventing JavaScript access and mitigating XSS-based token theft. CSRF protection is applied at the gateway middleware layer. All ML service traffic is proxied — the service is not externally routable.

🔐 bcrypt Password Hashing

Industry-standard adaptive hashing with configurable cost factor. Resistant to brute-force and rainbow table attacks.

🍪 HttpOnly JWT Cookies

Tokens are inaccessible to JavaScript execution context. Prevents the most common class of XSS token exfiltration attacks.

🛡️ CSRF Middleware

Cross-Site Request Forgery protection applied at the gateway layer. Validates request origin headers on all state-mutating endpoints.

⏱️ Rate Limiting

Per-IP throttling prevents abuse of auth endpoints and recommendation APIs. Configurable via gateway environment variables.

RECOMMENDATION ENGINE

Recommendation Engine: Standard Flow

The standard recommendation pipeline is a deterministic six-stage process. Recommendations are always produced by trained ML models operating over structured user-item interaction data — LLMs are not involved in this path. The pipeline is designed for low-latency inference with graceful fallback when filter constraints over-restrict the result set.

1 Client Request

User triggers a recommendation request from the frontend (e.g., `GET /api/recommend/anime`) with optional filter parameters such as genre, rating threshold, or episode count range.

2 Gateway Authentication

The API Gateway extracts and validates the JWT from the `HttpOnly` session cookie. Invalid or expired tokens receive a `401 Unauthorized` response before the request reaches the ML service.

3 Domain Service Inference

The gateway proxies the validated request to the appropriate domain service — `AnimeService` (SVD model), `SteamService` (ALS model), or `BookService` (baseline model). Each service applies domain-aware filtering before and after scoring.

4 Constraint Relaxation

If applied filters yield fewer results than the minimum threshold, `relax_constraints_and_retry` progressively removes the most restrictive constraints in order of strictness, retrying inference until a sufficient result set is produced or the baseline JSON fallback is invoked.

5 Metadata Enrichment

Raw ML output (item IDs and scores) is enriched with full metadata — titles, cover images, ratings, genres, and domain-specific fields — retrieved from MongoDB via the `MongoManager` connection pool with optimized batch queries.

6 Frontend Rendering

The enriched structured JSON response is consumed by the React frontend and rendered as `DomainProductCards` — domain-aware card components that display the appropriate metadata fields per domain type.

AI ASSISTANT

AI Assistant & Semantic Search

The AI assistant path extends the standard recommendation flow with natural language understanding and vector similarity search. Critically, the LLM is constrained to explanation and intent parsing roles only — item selection always originates from Pinecone vector search results grounded in real metadata. This architecture eliminates hallucinated recommendations while enabling conversational interaction.

End-to-End AI Flow

01

Natural Language Input

User submits a free-text query via the Chat panel (e.g., *"Show me dark fantasy anime with high ratings from 2020 onwards"*).

02

Intent Classification

HybridLLMClient sends the query to the active LLM provider with a structured prompt. The LLM extracts intent and outputs a JSON constraints object (genre, year range, rating floor, etc.).

03

Vector Embedding & Search

The query text is embedded using `SentenceTransformers` into a 384-dimension dense vector. Pinecone performs approximate nearest-neighbor search with Cosine similarity against the domain index.

04

LLM Explanation

The LLM generates a natural language explanation for each returned item using only the real metadata fields. **No-Fabrication Style Instructions** are enforced via system prompt to prevent hallucinated attributes.

HybridLLMClient Fallback Chain

1 Claude (Anthropic)

Final fallback provider. Invoked only if Groq is rate-limited or unavailable.

2 Groq

Secondary provider. Fast inference via Groq's LPU architecture. Activated on Gemini 429 errors.

3 Gemini (Google)

Primary provider. Handles the majority of intent parsing and explanation workloads under normal load.

Rate limit errors (`HTTP 429`) on any provider trigger automatic cascading to the next provider with no user-visible interruption.

DOMAIN ECOSYSTEM

Domain Ecosystem & ML Models

CompareX supports three independent recommendation domains, each with a domain-specific ML model, metadata schema, and Pinecone vector index. Model selection reflects the nature of available feedback signals: explicit rating data enables factorization-based SVD, while implicit behavioral data (playtime) is better served by ALS. All domains share a unified vector search interface via Pinecone.

📚 BookCrossing

Model: Baseline fallback (no collaborative filtering)

Feedback Type: Sparse explicit ratings

Metadata Fields: Title, author, publication year, Amazon cover images

Fallback Strategy: Curated static JSON list with popularity-ranked defaults when no personalized signal is available

Vector Search: Pinecone index for semantic book queries

🎮 Steam Games

Model: ALS — Alternating Least Squares (implicit feedback)

Feedback Type: Playtime hours (implicit behavioral signal)

Metadata Fields: Game title, themes, genres, developer, platform tags

Model Rationale: ALS excels at implicit feedback where absence of play does not indicate dislike — only playtime strength matters

Vector Search: Pinecone index for semantic game queries

🎌 Anime

Model: SVD — Singular Value Decomposition (explicit feedback)

Feedback Type: Numeric user ratings (explicit preference signal)

Metadata Fields: Title, studio, episode count, genre tags, structured ratings

Model Rationale: SVD performs matrix factorization over known rating values, making it optimal for dense explicit feedback datasets

Vector Search: Pinecone index for semantic anime queries

Model Comparison: ALS vs. SVD

Dimension	ALS	SVD
Feedback Type	Implicit (behavioral)	Explicit (ratings)
Input Signal	Playtime hours	Numeric user ratings
Absence Interpretation	Unobserved (not dislike)	Not rated (missing)
Applied Domain	Steam Games	Anime
Storage Format	Pickled model file	Pickled model file

Pinecone Vector Index Configuration

Parameter	Value
Embedding Model	SentenceTransformers
Vector Dimensions	384
Similarity Metric	Cosine
Index Coverage	All three domains
Query Type	Approximate nearest-neighbor (ANN)

LOCAL DEVELOPMENT

Local Development Setup

CompareX requires three concurrent terminal processes for full local operation. Each service runs on a fixed port. Start the ML service first — the gateway will fail health checks if the upstream inference service is unavailable on startup. All environment variables must be configured before launching any service.

Prerequisites

Requirement	Version / Spec
Node.js	v18 or higher
Python	3.10 or higher
MongoDB	Atlas cluster (cloud)
Pinecone Index	384 dimensions, Cosine metric
LLM API Keys	Gemini, Groq, Claude (at least one)

Environment Files

DEPLOYMENT

Deployment & Production

CompareX is deployed across two hosting platforms: Vercel for the static frontend and Render for the Node.js gateway and Python ML service. MongoDB Atlas and Pinecone require network access configuration to accept inbound connections from Render's dynamic IP ranges.

Frontend → Vercel

Deploy from the `/frontend` directory. Update `vercel.json` to configure reverse proxy rewrites from `/api/*` to the Render Gateway URL. Set `VITE_API_URL` environment variable in the Vercel project settings.

Backend Services → Render

Deploy gateway and ml-service as Render Web Services using the `render.yaml` Blueprint file. The Blueprint defines build commands, start commands, and environment variable references for both services in a single configuration.

Database Network Access

Configure MongoDB Atlas IP Access List to allow `0.0.0.0/0` (all IPs) to accommodate Render's dynamic egress IPs. Apply the same open access policy to Pinecone index settings. Restrict in production using VPC peering where compliance requires it.

render.yaml Blueprint Structure

```
services:
  - type: web
    name: comparex-gateway
    env: node
    buildCommand: npm install
    startCommand: npm start
    envVars:
      - key: MONGODB_URI
        sync: false
      - key: JWT_SECRET
        sync: false
      - key: ML_SERVICE_URL
        value: https://comparex-ml.onrender.com

  - type: web
    name: comparex-ml
    env: python
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn app.main:app --host
0.0.0.0 --port 8001
```

vercel.json Proxy Configuration

```
{
  "rewrites": [
    {
      "source": "/api/:path*",
      "destination": "https://comparex-
gateway.onrender.com/api/:path*"
    }
  ]
}
```

Production Checklist

- [x] ~~Deploy ML service to Render first~~
- [x] ~~Copy ML service URL into gateway environment variable~~
- [x] ~~Deploy gateway to Render with all env vars set~~
- [x] ~~Update vercel.json with Render gateway URL~~
- [x] ~~Deploy frontend to Vercel from /frontend directory~~
- [x] ~~Open MongoDB Atlas network access to 0.0.0.0/0~~
- [x] ~~Open Pinecone index network access~~

Live Demo: recommendation-system-six-nu.vercel.app